

Promoting `discovery_okuda` into `src/plexus`

the 3D vertex model becomes part of the language, and what had to be true for that

1. What this is

`discovery_okuda` held the 3D active vertex model, reaction–diffusion on the cell graph, the junction and medioapical myosin pools, and the whole basement-membrane / extracellular-matrix stack: 52 registered operator names across 21 files, and behind them `log/okuda` and `log/okuda_ECM`. None of it was in the framework `paper/plexus2.tex` describes. The core's own `src/plexus/operators/` carried 43 names in 45 one-operator files, and not one mesh or chemistry operator among them.

This note is the record of moving that vocabulary into the core, written as each phase landed rather than summarised afterwards. Its structure follows the paper's four parts — mechanism, equations and symbols, the Plexus2 container, gates — with one section per phase, and its gate tables are generated from the run record by `tools/promotion_note.py` rather than transcribed.

The two things that had to be true. A promotion of this size is only checkable if it can be shown to change nothing. Two gates run on every phase.

1. **The twin run.** The spec is run in `okuda` *first*, and its twin is run in the codebase; previous `okuda` results are never the reference, because `okuda` is still moving. Both sides are submitted *together*, as two `bsub` jobs on `gpu_14`, and the comparison is array by array, bit for bit, reduced to one `sha1` per side.
2. **The `okuda_ECM` gates 00–04**, each assigned to the phase that enables it, each with a table of thresholds declared *before* the promoted run. §5.

2. Phase 0 — the engine

2.1 The half-edge mesh becomes an engine concept

A closed cellular surface is stored as a flat half-edge table: three parallel `int64` arrays `E_srce`, `E_trgt`, `E_face`, plus the counts `nF` (faces = cells) and `Nv`. Face f 's ring is recovered by walking the table and appending `E_srce[k]` for every k with `E_face[k] == f`, *in table order*. That ordering is the geometry: `rings_from_flat_3d` never reads `E_trgt`, never sorts and never validates, so a sort, a compaction, or a scatter-into-reservoir by any future writer produces garbage rings in silence.

Before this promotion the table was a private attribute one operator invented — `grep -rn "_mesh" src/plexus/` returned nothing at all — while roughly forty consumers read it: the chemistry, the mechanics, the TIs, the cross-scale contact operators, the recorder, the salvage, the instrument's fingerprint, the renderers. A spec could schedule `cell_mechanics` without `mesh_seed` and die of `AttributeError` from inside an autograd pass.

It is now declared and allocated:

<code>sets: {vertex: {mesh: half_edge, cell_set: cell}}</code>	the declaration
<code>plexus.models.mesh.MeshTable</code>	the table, a <code>dict</code> subclass
<code>plexus.models.topology</code>	its algorithms (rings, edge split, face division)
<code>engine._build_mesh</code>	allocates an <i>empty</i> table in <code>build</code> 's pass 1
<code>seed_mesh</code>	<i>fills</i> it, rather than rebinding a new one

Why it subclasses `dict`, and this is not a style choice. `discovery_okuda/instrument.py:43` reads `if isinstance(m, dict):`. That is the acted ledger — the thing that decides whether an operator *did anything* — wrapped in a bare `except Exception: return ()`. A `MutableMapping`

that is not a dict subclass fails that test silently, the fingerprint collapses to `()`, and every operator whose only effect is on the mesh (`cell_grow` writing `Vof`, `cell_divide` changing `nF`, `junction_myosin` writing `myo`) scores zero acts. The campaign then stamps `valid_evidence: False` on runs that were fine.

And the namespace stays open, deliberately. Five keys are reserved; everything else is an extension namespace, because three live mechanisms depend on a name being *absent*: `cell_shape_probe` publishes a field and then deletes it (absent means “no cell dies”; zero-filled would mean “kill everything”), `ecm_gate_growth`’s entire entry condition is `'mg_scale'` in `m`, and the apoptosis discriminator resolves whatever name its spec gives it.

The carry was written four times. Permuting every per-face array through `keep` is one operation, and it existed in `cell_divide`, in `cell_die`, in `_carry_face_state` and in `edge_flip` — and the fourth copy silently omitted the medioapical myosin, because a per-face state added by one operator has to be known to every topology operator and that one did not get the memo. It is now `MeshTable.reindex_faces`, and the renumber of the paired *cell* set is `Hierarchy.renumber_set`, which moves all four stores the engine keeps per set. The fourth, `_delta_blocks`, is the one both hand-rolled versions missed: they guarded on `isinstance(key, tuple)` while `add_delta` keys by level *name*. It is inert today only because `chem` happens to be the cell set’s coordinate block, so its deltas travel in `_delta`; a second `first_order` block on that set makes it live.

2.2 The recorder keeps the topology

`topo_record` appended its history *inside* the mesh table (`m.setdefault("hist", [])`), so the structure the engine now owns grew without bound for the length of the run and every consumer walked past a list of every frame it had ever had. The engine records it: one `MeshTable.snapshot()` per recorded row, written to both `zarr` and `npz` as a ragged store with *no pickle* — the three half-edge columns concatenated, plus row offsets, plus `nF/Nv` per row. Euler now computes off the core’s own trajectory file,

$$V - E + F = Nv[t] - \frac{1}{2}(\text{off}[t+1] - \text{off}[t]) + nF[t] = 2,$$

which is what gate 00’s closed-form row needs and what `topo_record` could not give it.

2.3 Two bugs that destroyed finished work

- `Group.create_dataset(name, data=...)` is the `zarr-2` spelling. `zarr 3` kept the method, deprecated it, and made `shape` a *required* keyword, so every core generation on the cluster (`zarr 3.1.5`) died *after* the whole simulation had run and *before* the `npz` was written — 25 minutes of GPU, an empty `simulation.zarr`, no trajectory. The devcontainer has `zarr 2.18.7`, which is why it never appeared in development.
- The `npz` writer put `d["pos"]` in unconditionally. A mesh model’s `cell` set has no `pos` block at all — it carries `chem`, `cen`, `area` — so `None` went in as a 0-d *object* array and every read of the file died on `allow_pickle`. Worse, the recorded blocks were not written at all: a reaction–diffusion run wrote a trajectory with no reaction in it.

2.4 325 of 461 specs did not load

Not “would eventually fail”: `ValueError: operator 'seed_mesh' not in registry`, for every `r0*` the campaign had ever written. The specs had been migrated to the post-seed-refactor `seed_<noun>` convention — the one core already uses for `seed_from_segmentation` — and the `okuda` classes had not, so the corpus split 324/137 across two spellings of one operator and the larger half was dead. Both spellings resolve now: canonical `seed_mesh` and `seed_cell_chem`, aliases `mesh_seed` and `cell_chem_seed`. 460 of 461 load; the one that does not names `morphogen_growth_3d`, which is stale in a different way and is recorded rather than guessed at.

3. Phases A–D — the operators

`tools/group_operators.py` moved them, and the choice to write a script rather than edit by hand is the reason bit-equality is a reachable claim across 3,800 lines. It hoists each source’s imports, rewrites the ones that named a sibling by bare module name, and concatenates the bodies under a banner. Nothing inside a function body is touched. `-verify` compares *text*: every top-level definition’s exact source, decorators included, searched for as a substring of the module it landed in.

That `verify` earned itself on the first run. `ClassDef.lineno` is the line of the `class` keyword, not of the `@register_operator(...)` above it, so taking it as the start of the body dropped the decorator off the *first* class in every file: `cell_geometry` silently left the registry and 461 specs stopped loading. The counting check — “70 names, looks right” — had passed.

from	to	lines
<code>topology_ops.py</code>	<code>src/plexus/models/topology.py</code>	218
<code>mesh_ops + t1_ops + monolayer_ops</code>	<code>operators/vertex_ops.py</code>	2183
<code>chem_ops + shape_chem_ops + shape_probe_ops</code>	<code>operators/diffusion_reaction.py</code>	1490
<code>junction_ops + medioapical_ops</code>	<code>operators/junction_ops.py</code>	856
<code>ecm_ops + block_ops</code>	<code>operators/ecm_ops.py</code>	864
<code>membrane_ops + integrin_ops</code>	<code>operators/membrane_ops.py</code>	2187
<code>mesh_contact + bm_sense + plate + surface + load</code>	<code>operators/contact_ops.py</code>	1183

The shims are load-bearing. Thirty files import `mesh_ops` / `chem_ops` / `topology_ops` by bare module name — `run_one.py`, `instrument.py`, `vtk_render.py`, `metrics.py` and twenty archive and analysis scripts — and the campaign is still running against them. Each moved file is a re-export, *private names included*: `t1_ops` calls `_carry_face_state` and `analyze_forces` reaches for `_engine_owns_clock`.

Two operators did not come. `mpm_boundary` and `bm_strain` are rejected by `AUDIT.md`, and a rejection that lives only in a markdown file is one the next reader re-promotes by accident. Their classes are *cut out* of the module that moved and left in `discovery_okuda/ops/membrane_ops.py` under a banner saying why: an archived spec that names one still runs, and no spec can reach either from the core registry. `mpm_boundary` overwrites grid-node velocity, so the constraint is kinematic — momentum not conserved, the reaction discarded — and its standoff is set by the B-spline stencil width, measured across `recover` 0/2/6/20 as 46.6%/3.8%/11.5%/13.9% of the sheet inside the tissue against standoffs +0.0006/ +0.0124/ +0.0088/ +0.0069, never reaching the $0 \rightarrow +0.002$ that would mean “just touching”. `integrin_track` is the constraint it should have been. `bm_strain` is, in `AUDIT`’s words, “not a mechanism”.

3.1 Redundancy: the four merge candidates, settled by reading them

- `interface_tension + interface_push` — **do not merge**. They were one operator, `rd_interface_tension`, carrying $K_{\text{purse}} \sum_{\text{iface}} \ell_e - K_{\text{extrude}} \sum_{\text{red}} ar$. The first term is ordinary vertex-model physics; the second does not model a force, it pays the tissue to produce the morphology the campaign was searching for. One name over both cost four rounds: `K_extrude` measured 0.0 in all 78 specs that ever carried the operator, so nothing was ever forced, and the `Grounder` still reported a round as “the same extrude-forced star for a fourth round” on three runs whose specs contain no such operator. The split *is* the fix.
- `ecm_stress` / `block_stress` — **merge, in Phase D**. Same body. They differ in two defaults — `scale` 0.15 for the matrix against 0.004 for the block, which is right, since the block is $\sim 130\times$ stiffer — and in *which module-level history list* they append to. That list is the only reason the duplicate exists.

- `ecm_seed / block_seed` — **do not merge**. Box-minus-a-cavity with aligned fibres against two slabs of jittered lattice. Same family, different geometry.
- `integrin_adhesion` vs `integrin_pull/seed/track` — **do not merge**. One is membrane→epithelium, the other matrix→membrane. The shared prefix is the problem, not the code; they go in one module with that sentence at the top of it.

3.2 The inventory

module	operator	family	kind	set	implementations / alias
<code>activation_pulse.py</code>	<code>activation_pulse</code>	fields	field	field	
<code>active_force.py</code>	<code>active_force</code>	mechanics	exchange	particle	alias
<code>active_stress.py</code>	<code>active_stress</code>	mechanics	exchange	particle	pulse_to_contraction alias pulse_to_active_stress
<code>agent_divide.py</code>	<code>agent_divide</code>	population	structural	cell	
<code>agent_gather.py</code>	<code>agent_gather</code>	coupling	exchange	cell	alias mpm_to_agent
<code>agent_grow.py</code>	<code>agent_grow</code>	population	structural	cell	
<code>agent_remodel.py</code>	<code>agent_remodel</code>	coupling	exchange	cell	
<code>agent_scatter.py</code>	<code>agent_scatter</code>	coupling	exchange	cell	alias agent_to_mpm
<code>aggregate.py</code>	<code>aggregate</code>	hierarchy	aggregate	cell	
<code>attraction_repulsion.py</code>	<code>attraction_repulsion</code>	interaction	lateral	particle	
<code>attractor_flow.py</code>	<code>attractor_flow</code>	motion	lateral	particle	
<code>bounce.py</code>	<code>bounce</code>	boundary	lateral	cell	
<code>broadcast.py</code>	<code>broadcast</code>	hierarchy	broadcast	particle	
<code>chemotax.py</code>	<code>chemotax</code>	fields	exchange	particle	
<code>cohesion.py</code>	<code>cohesion</code>	interaction	lateral	particle	
<code>contact_ops.py</code>	<code>bm_sense</code>	signalling	structural	vertex	
<code>contact_ops.py</code>	<code>ecm_gate_growth</code>	population	structural	vertex	
<code>contact_ops.py</code>	<code>ecm_load</code>	mechanics	structural	vertex	
<code>contact_ops.py</code>	<code>mesh_contact</code>	boundary	lateral	particle	
<code>contact_ops.py</code>	<code>mesh_inside</code>	hierarchy	lateral	particle	
<code>contact_ops.py</code>	<code>plate_confine</code>	boundary	structural	vertex	
<code>contact_ops.py</code>	<code>surface_track</code>	hierarchy	structural	particle	
<code>decay.py</code>	<code>decay</code>	fields	field	field	
<code>deposit.py</code>	<code>deposit</code>	fields	exchange	cell	
<code>diffuse.py</code>	<code>diffuse</code>	fields	field	field	finite_difference, spectral
<code>diffusion_reaction.py</code>	<code>cell_chem_diffuse</code>	fields	lateral	cell	graph_laplacian, interface_weighted
<code>diffusion_reaction.py</code>	<code>cell_chem_from_shape</code>	fields	lateral	cell	apical_area, curvature, pressure, tension
<code>diffusion_reaction.py</code>	<code>cell_chem_react</code>	fields	lateral	cell	brusselator, gierer_meinhardt, gray_scott
<code>diffusion_reaction.py</code>	<code>cell_geometry</code>	hierarchy	aggregate	cell	
<code>diffusion_reaction.py</code>	<code>cell_grow</code>	population	structural	vertex	balance, default, sizer, timer
<code>diffusion_reaction.py</code>	<code>cell_neighbours</code>	topology	rewire	cell	
<code>diffusion_reaction.py</code>	<code>cell_shape_probe</code>	hierarchy	lateral	cell	aspect, shape_index
<code>diffusion_reaction.py</code>	<code>interface_push</code>	mechanics	lateral	vertex	
<code>diffusion_reaction.py</code>	<code>interface_tension</code>	mechanics	lateral	vertex	
<code>diffusion_reaction.py</code>	<code>seed_cell_chem</code>	seed	seed	cell	alias cell_chem_seed
<code>drag.py</code>	<code>drag</code>	motion	lateral	particle	
<code>ecm_ops.py</code>	<code>block_seed</code>	seed	seed	particle	
<code>ecm_ops.py</code>	<code>block_stress</code>	hierarchy	lateral	particle	
<code>ecm_ops.py</code>	<code>cell_exclude</code>	boundary	structural	particle	
<code>ecm_ops.py</code>	<code>ecm_from_cell</code>	mechanics	lateral	particle	replay, sphere
<code>ecm_ops.py</code>	<code>ecm_seed</code>	seed	seed	particle	
<code>ecm_ops.py</code>	<code>ecm_stress</code>	hierarchy	lateral	particle	
<code>glide.py</code>	<code>glide</code>	motion	lateral	cell	
<code>graph.py</code>	<code>radius_graph</code>	topology	rewire	particle	
<code>gravity.py</code>	<code>gravity</code>	mechanics	lateral	cell	
<code>junction_ops.py</code>	<code>cytokinetic_ring</code>	mechanics	structural	vertex	
<code>junction_ops.py</code>	<code>junction_myosin</code>	mechanics	structural	vertex	default, two_pool
<code>junction_ops.py</code>	<code>junction_sync</code>	mechanics	rewire	vertex	
<code>junction_ops.py</code>	<code>medioapical_myosin</code>	mechanics	lateral	cell	
<code>material_map.py</code>	<code>apply_material_map</code>	mpm	exchange	particle	
<code>membrane_ops.py</code>	<code>adhesion_pull</code>	mechanics	exchange	particle	
<code>membrane_ops.py</code>	<code>adhesion_seed</code>	seed	seed	particle	

module	operator	family	kind	set	implementations / alias
membrane_ops.py	adhesion_turnover	topology	rewire	particle	
membrane_ops.py	bm_bond	mechanics	lateral	particle	
membrane_ops.py	bm_contact	boundary	lateral	particle	
membrane_ops.py	bm_crosslink	topology	rewire	particle	
membrane_ops.py	bm_remodel	population	lateral	particle	
membrane_ops.py	bm_repel	boundary	lateral	particle	
membrane_ops.py	bm_secrete	population	structural	particle	
membrane_ops.py	bm_seed	seed	seed	particle	
okuda membrane_ops.py	bm_strain	mpm	lateral	particle	
membrane_ops.py	bm_unbond	topology	rewire	particle	
membrane_ops.py	integrin_adhesion	mechanics	lateral	particle	
membrane_ops.py	integrin_pull	mechanics	lateral	particle	
membrane_ops.py	integrin_seed	seed	structural	particle	
membrane_ops.py	integrin_track	mechanics	structural	particle	
okuda membrane_ops.py	mpm_boundary	boundary	field	field	
mpm.py	mls_mpm_mechanics	mpm	exchange	particle	
mpm_anchor.py	mpm_anchor	mechanics	lateral	particle	
mpm_gather.py	mpm_gather	mpm	exchange	particle	alias g2p
mpm_grid_update.py	mpm_grid_update	mpm	field	field	
mpm_scatter.py	mpm_scatter	mpm	exchange	particle	alias p2g
mpm_spin.py	mpm_spin	mechanics	lateral	particle	
mpm_strain.py	mpm_strain	mpm	lateral	particle	
pacemaker.py	pacemaker	fields	field	field	
polarity_align.py	polarity_align	polarity	exchange	cell	alias heading_align
polarity_flow_align.py	polarity_flow_align	polarity	exchange	cell	alias flow_align
prescribed_field.py	playback	harness	field	field	
sediment.py	sediment	motion	lateral	cell	
segmentation_seed.py	seed_from_segmentation	seed	seed	particle	
sense.py	sense	signalling	exchange	cell	
separation.py	separation	interaction	lateral	particle	
signal.py	signal	signalling	lateral	neuron	
squared_law.py	squared_law	interaction	lateral	particle	
stillerer_weber.py	stillerer_weber	interaction	lateral	particle	autograd
velocity_align.py	velocity_align	interaction	lateral	particle	alias alignment
velocity_cruise.py	velocity_cruise	motion	lateral	particle	alias cruise
vertex_ops.py	cell_die	population	structural	vertex	
vertex_ops.py	cell_divide	population	structural	vertex	default, doubler, timer
vertex_ops.py	cell_mechanics	mechanics	lateral	vertex	default, monolayer
vertex_ops.py	edge_flip	topology	rewire	vertex	
vertex_ops.py	seed_mesh	seed	seed	vertex	alias mesh_seed
vertex_ops.py	topo_record	harness	structural	vertex	

91 of 93 canonical operator names resolve into `src/plexus/`. The remainder are the two AUDIT.md rejects, registered in `discovery_okuda` only.

4. The twin-run gate

The protocol, stated exactly. *Run the spec in okuda first, and run the twin spec in the codebase. Do not use previous okuda results — they can be updated. Run both on the L4 cluster node, in parallel.* Nothing already sitting in `log/okuda` is ever the reference: those artefacts were produced by whatever okuda was on the day they were written, and `round.py` runs campaigns into that tree while `staged.py` writes into it by hand. A promotion checked against a stale file proves that the file has not changed, which is not the question.

What counts as identical. The two writers differ — okuda’s `RunArchive` writes `traj.npz` (`pos_i`, `act_i`), the core writes `trajectory.npz` (`<set>__pos`, `<set>__occ`, `<set>__chem`) — so a file hash cannot match and is not the test. What makes the two comparable at all is that okuda’s own frames come out of `engine._assemble`: `run_one.py:593` reads out `["sets"]["vertex"]["pos"]` and out `["sets"]["cell"]["state"]["chem"]`, the same structure the core writes. So okuda’s `pos_i` is the core’s `vertex__pos[t]` cropped to the live prefix, and `act_i` is `cell__chem[t][:nF,0]`. okuda keeps the ~60 frames its movie draws and stores their row indices in `ticks`; the core keeps every row; the okuda side is read first and its `ticks` select the core’s rows.

The repeatability floor is zero, and it was measured. Byte-identity is only the right criterion if the platform can deliver it, and over 1,800 frames that is not obvious: one differing mantissa bit can move an `edge_flip` decision ($\ell < 1_{\text{th_frac}} \cdot \bar{\ell}$) and from then on the two runs have *different topology*, at which point “max $|\Delta|$ ” is not small, it is meaningless. So `-phase R` runs the identical spec, the identical code and the identical commit *twice*, as two separate jobs. `b_star` at 1,800 frames, growing 2,000 cells to 12,272 through division and T1: `beb6fb24fe86dcde` both times, $\max|\Delta| = 0.0$ exactly, no first differing bit, over 120 arrays per run. The floor is zero, so `tol = 0` is a real criterion at full length and every row keeps it. A tolerance would have to be justified by a floor above zero; there is not one.

Determinism is asserted, not assumed. `engine.py` sets `torch.use_deterministic_algorithms(True, warn_only=True)`, and `warn_only` silently downgrades any kernel with no deterministic implementation — which is exactly where two runs of one spec stop matching. Each side runs with `PLEXUS_STRICT_DETERMINISM=1`, which turns the downgrade into an exception, so a side that dies of it *fails* rather than passing on a comparison it never earned.

phase	spec	frames	A	B	digest	outcome
0	<code>apop_many</code>	600	<code>okuda@HEAD</code>	<code>okuda</code>	<code>0c0b38ec027ca47d</code>	identical
0	<code>apop_one</code>	600	<code>okuda@HEAD</code>	<code>okuda</code>	<code>568eaeab77720645</code>	identical
0	<code>apop_patch_big</code>	600	<code>okuda@HEAD</code>	<code>okuda</code>	<code>ac5b5b85075cbf55</code>	identical
0	<code>apop_rings9</code>	600	<code>okuda@HEAD</code>	<code>okuda</code>	<code>cbf719d02511d058</code>	identical
0	<code>apopgeo_half</code>	600	<code>okuda@HEAD</code>	<code>okuda</code>	<code>1f9d217766575360</code>	identical
0	<code>b_null_plain</code>	1800	<code>okuda@HEAD</code>	<code>okuda</code>	<code>14aab859c575fe25</code>	identical
B-core	<code>b_gs_plain_soft_lo</code>	1800	<code>okuda</code>	<code>core</code>	<code>906afc1a3931063a</code>	identical
B-core	<code>b_star</code>	1800	<code>okuda</code>	<code>core</code>	<code>beb6fb24fe86dcde</code>	identical
R	<code>b_null_plain</code>	1800	<code>okuda</code>	<code>okuda</code>	<code>14aab859c575fe25</code>	identical
R	<code>b_star</code>	1800	<code>okuda</code>	<code>okuda</code>	<code>beb6fb24fe86dcde</code>	identical

10 of 10 rows identical.

Every row runs at its spec’s own length. 100 frames was too short to test anything structural: `b_star`’s topology keeps changing past frame 800, so a 100-frame gate green-lights a promotion that has not run one of the divisions and flips it claims to preserve.

Which row is the actual claim. The `okuda@HEAD` vs `okuda` rows compare `okuda` before the change to `okuda` after it: they prove the *move* changed no bytes, which is necessary and is not the question. The B-core rows are the claim — `Plexus_Main.py -o generate` against `run_one.py`, different runner, different recorder, different writer, the core registry with no `okuda` import on one side. `b_star`’s B-core digest is the same hex the repeatability pair produced, so the core run, the `okuda` run and two independent repeats of the `okuda` run all agree bit for bit, positions and chemistry alike.

Four of the rows are the minisite’s death scenes , run to their own length rather than truncated, because a gate whose archive stops before the deformation cannot show what it claims to test. A 45° cap of 293 cells draws the surface *inward* — an invagination, the one deformation growth cannot make; nine bands of 895 deaths close over every gap and the vesicle stays a sphere at reduced volume 0.981; 285 of 400 cells above the equator leave an ellipsoid pulled from a sphere at `gyr_prolate` 1.869, with the topology intact. `apop_many` is the only new spec in the set, and it exists because one row moved is not the hard case for a renumber: its 200 deaths are spread along the Fibonacci spiral by a stride of 10, so `keep` is interleaved rather than a contiguous block — the case a renumber written as a truncation passes and one written as a gather has to get right.

5. The okuda_ECM gates 00–04

What they were. Not assert-scripts. `test_00_spheroid.py` writes a `spec.yaml` saying what it did, a `metrics.json` and some plots — it *reports*. Its “spec” is built imperatively in Python, and its output lands wherever the script chose under `log/okuda_ECM/`. And not one of the 461 specs in `config/okuda/` names a single junction, ECM, membrane or contact operator, so 34 of the operators just promoted have no declared spec at all.

What they become. Two dedicated homes, and a threshold that exists before the run.

- `config/gates/` — one yaml per gate, lifted out of the script, carrying the sets, the operators, the schedule, the frame count, the seed, *and* the declared thresholds with their tier. The spec becomes the gate’s definition rather than a side effect of running it, which is what makes a threshold auditable before the run.
- `log/gates/<gate>/` — the promoted run’s outputs: `metrics.json`, the gate table as written, the plots, the movie. `log/okuda_ECM/` stays as the okuda side and is not written over, so the two can always be diffed.

The tiers are the paper’s. *Bookkeeping* asks whether the code does what the operator says. *Closed form* asks whether the implementation reproduces the physics it was *given*. *Measurement* asks whether the model agrees with something observed in cells — and that tier is only available once the model declares its units, because before a `units:` block a comparison with a measurement is a comparison between a number and a quantity. None of the okuda specs declares units. That is stated here rather than worked around, and it bounds what this gate table can claim.

The okuda_ECM gates have not yet been run from the promoted registry; this table is written by `tools/run_gates.py`.

6. Status

phase	what	state
0	mesh as an engine concept; recorder; determinism; live snapshot	done
A	OKUDA_PROMOTION.md: 50 names, contracts, merge verdicts	done
B	<code>vertex_ops.py</code> , <code>diffusion_reaction.py</code> , <code>models/topology.py</code>	done
B-core	the core runs the model with no okuda import, bit-identical	done
C	<code>junction_ops.py</code>	done
D	<code>ecm_ops.py</code> , <code>membrane_ops.py</code> , <code>contact_ops.py</code>	done
0b	MPM regrouped into <code>mpm_ops.py</code>	not started
gates	00–04 lifted into <code>config/gates/</code> , run into <code>log/gates/</code>	in flight
E	regroup the remaining core; VTK as the engine’s renderer	not started
F	the three minisite scenes under <code>config/atlas/</code>	not started
G	replication: <code>r010_00_ctrl</code> and <code>b_star</code> as twin runs	not started